



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Refatoração de Code Smells no Projeto Great Tables



Francisco Samuel
fsamuel@alu.ufc.br



Wesley Pinheiro
wesleylpinheiro@gmail.com



Contexto

- **Objetivo:** aplicar refatoração com auxílio de LLM em um projeto open source Python.
- **Ferramentas:** Pylint (identificação), Radon/CodeCarbon/Pytest (métricas), ChatGPT 5.4 Mini (LLM para refatoração).

Projeto Analisado

O projeto analisado foi o Great Tables, uma biblioteca Python que permite criar tabelas visualmente incríveis a partir de componentes estruturais coesos.

O problema que resolve: Transforma dados brutos em relatórios e apresentações profissionais de forma simples, superando as limitações de formatação de ferramentas tradicionais.

Possui uma API intuitiva que separa a estrutura dos dados da sua apresentação final.





Projeto Analisado

- Estruturação modular: flexibilidade para combinar cabeçalhos, rodapés, colunas e rótulos de linha.
- Organização avançada: suporte para agrupar colunas sob rótulos superiores (spanner labels).
- Formatação poderosa: recursos robustos para estilizar e formatar os valores internos das células.



Coleta de métricas – Testes e Sustentabilidade

File	Statements	Missing	Excluded	Coverage
great_tables/_databackend.py	34	4	0	88.23529412
great_tables/_export.py	140	77	0	45
great_tables/_footnotes.py	24	4	0	83.33333333
great_tables/_formats.py	1396	107	0	92.33524355
great_tables/_formats_vals.py	116	24	0	79.31034483
great_tables/_gt_data.py	719	26	0	96.38386648
great_tables/_heading.py	6	0	0	100
great_tables/_helpers.py	246	5	0	97.96747967
great_tables/_locale.py	124	3	0	97.58064516
great_tables/_locations.py	374	12	0	96.79144385
great_tables/_modify_rows.py	88	0	0	100

Métrica CodeCarbon

Resultado Antes da Refatoração

Tempo de Execução

1.55 segundos

Emissões de Carbono

3.89E-07

Energia Consumida

3.95E-06 kWh

- Garantia de Qualidade (Pytest): A suíte de testes do repositório é robusta. Na execução inicial, 2.086 testes passaram e apenas 3 falharam, com arquivos centrais chegando a 100% de cobertura.
- Sustentabilidade (CodeCarbon): A execução da bateria de testes registrou um consumo de energia base de 3.95E-06 kWh em 1.55 segundos.



Coleta de métricas – Complexidade (Radon)

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
great_tables/_utils_nanoplots.py	function	_generate_nanoplot	F	148
great_tables/_utils_render_html.py	function	create_columns_component_h	F	78
great_tables/_utils_render_html.py	function	_create_row_component_h	F	70
great_tables/_utils_render_html.py	function	create_body_component_h	F	53
great_tables/_spanners.py	function	<u>tab_spanner</u>	D	27

- Complexidade Ciclométrica Crítica: O arquivo _utils_nanoplots.py apresentou a pior nota possível (Rank F), com complexidade de 148 na função _generate_nanoplot.

- Tamanho do Projeto: A base de código possui densidade considerável, com mais de 31.000 linhas de código (LOC) no total.

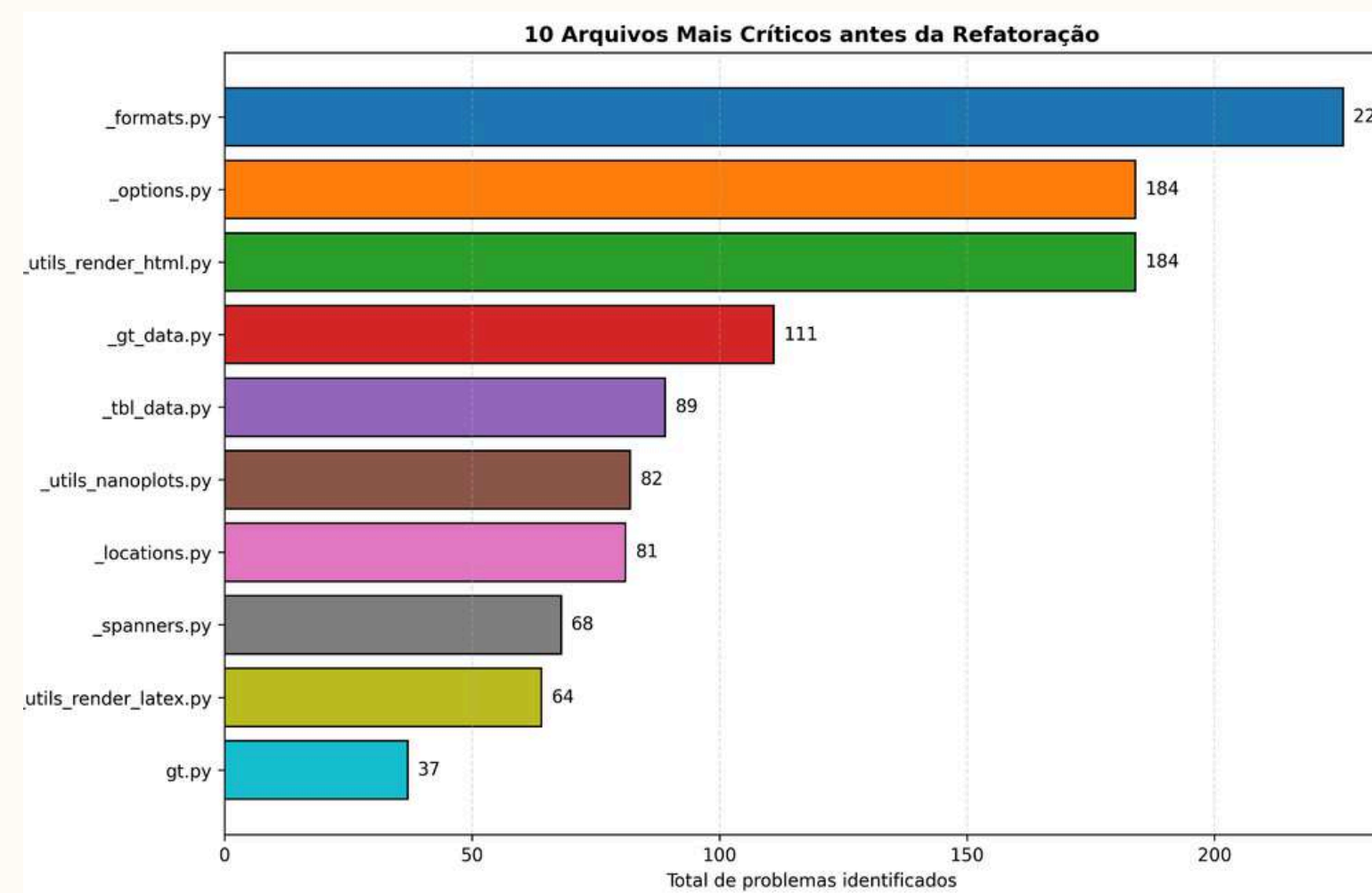
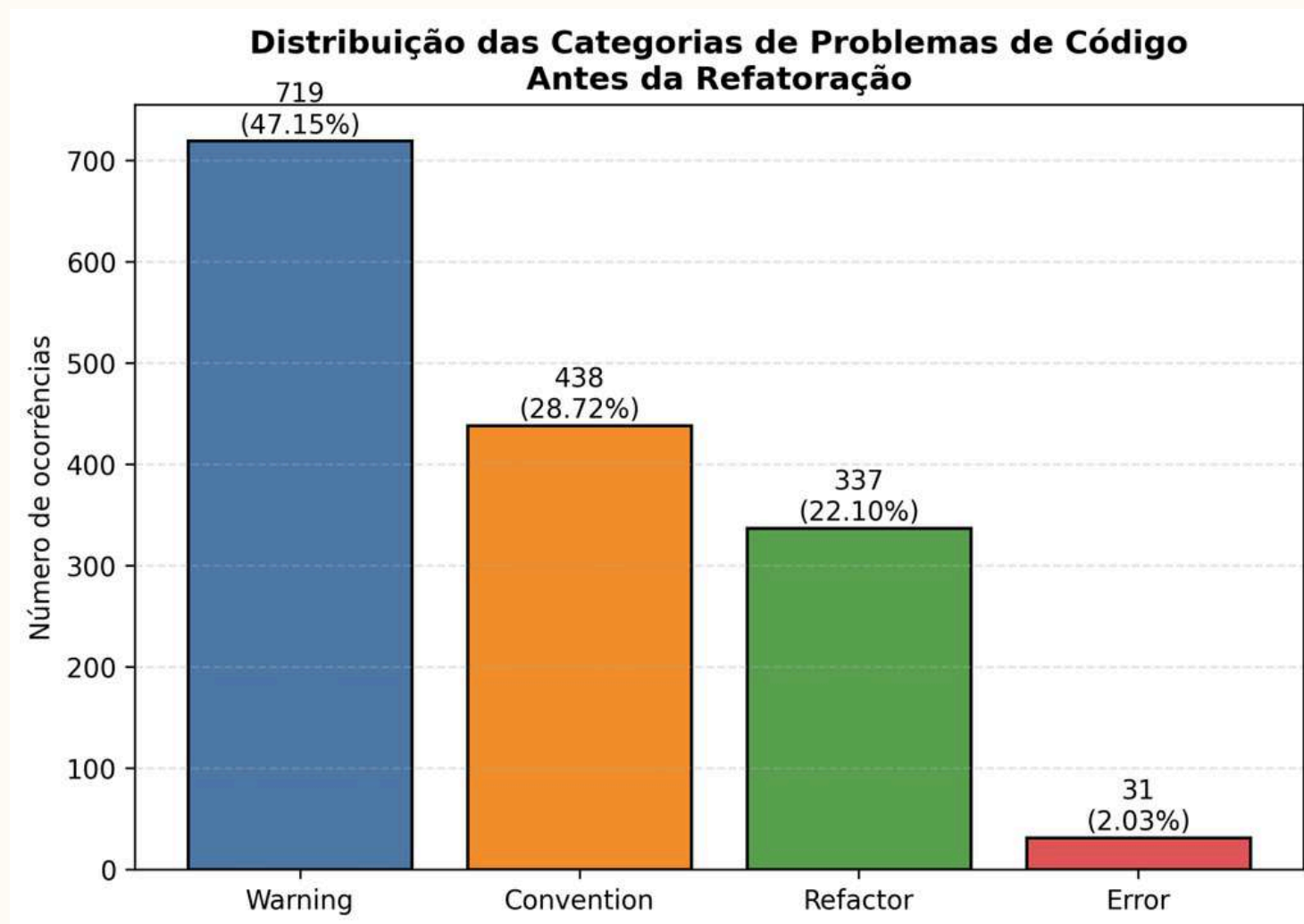
Tabela de piores métricas

arquivo	mi	rank_mi
great_tables/_spanners.py	6.1282	C
great_tables/_utils_nanoplots.py	5.0132	C
great_tables/_gt_data.py	2.7879	C
great_tables/_formats.py	0	C
great_tables/_utils_render_html.py	0	C

- Manutenibilidade Comprometida: Arquivos essenciais como _formats.py e _utils_render_html.py apresentaram Índice de Manutenibilidade 0 (Rank C).



Identificação dos Problemas (Visão Geral)



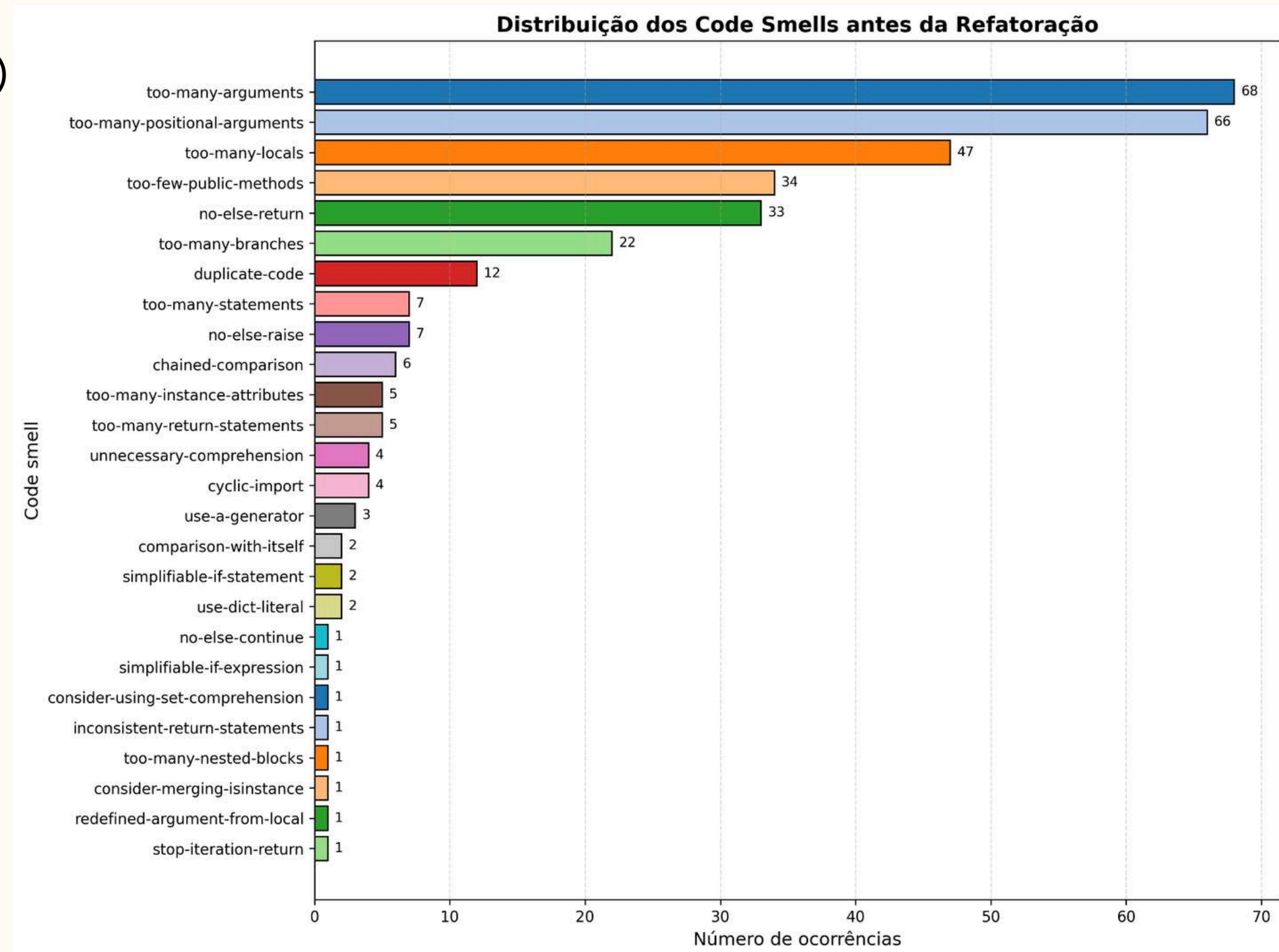
- A varredura do Pylint atribuiu ao código original a nota 7.76/10.
- Identificamos que a grande maioria dos problemas apontados no sistema são do tipo Warning (719 ocorrências) e Convention (438 ocorrências).
- Os arquivos _formats.py (226 problemas) e _options.py (184 problemas) destacaram-se como os maiores ofensores à qualidade do código.

Your code has been rated at
7.76/10



Foco da Refatoração (Os Code Smells)

- Segue a quantidade de ocorrências dos 3 tipos de code-smells analisados:
 - too-many-branches (22 ocorrências)
 - too-many-statements (7 ocorrências)
 - too-many-instance-attributes (5 ocorrências)
- Modelo de IA: ChatGPT 5.4 Mini.





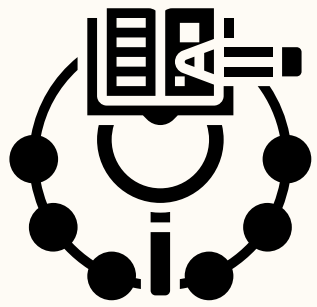
Pós-Refatoração – Redução de Complexidade (Radon)

- Pulverização de Fluxos: Funções críticas tiveram sua complexidade reduzida radicalmente com a extração para helpers menores.
- O pior caso anterior (`_generate_nanoplot`, com CC de 148) foi fragmentado. O teto atual de complexidade agora é 34 (na função `_create_columns_component_h_with_spanners`).
- Índice de Manutenibilidade (MI): Apesar das melhorias locais, arquivos centrais (`_gt_data.py`, `_formats.py`) ainda mantêm MI em 0, evidenciando que problemas arquiteturais profundos persistem.



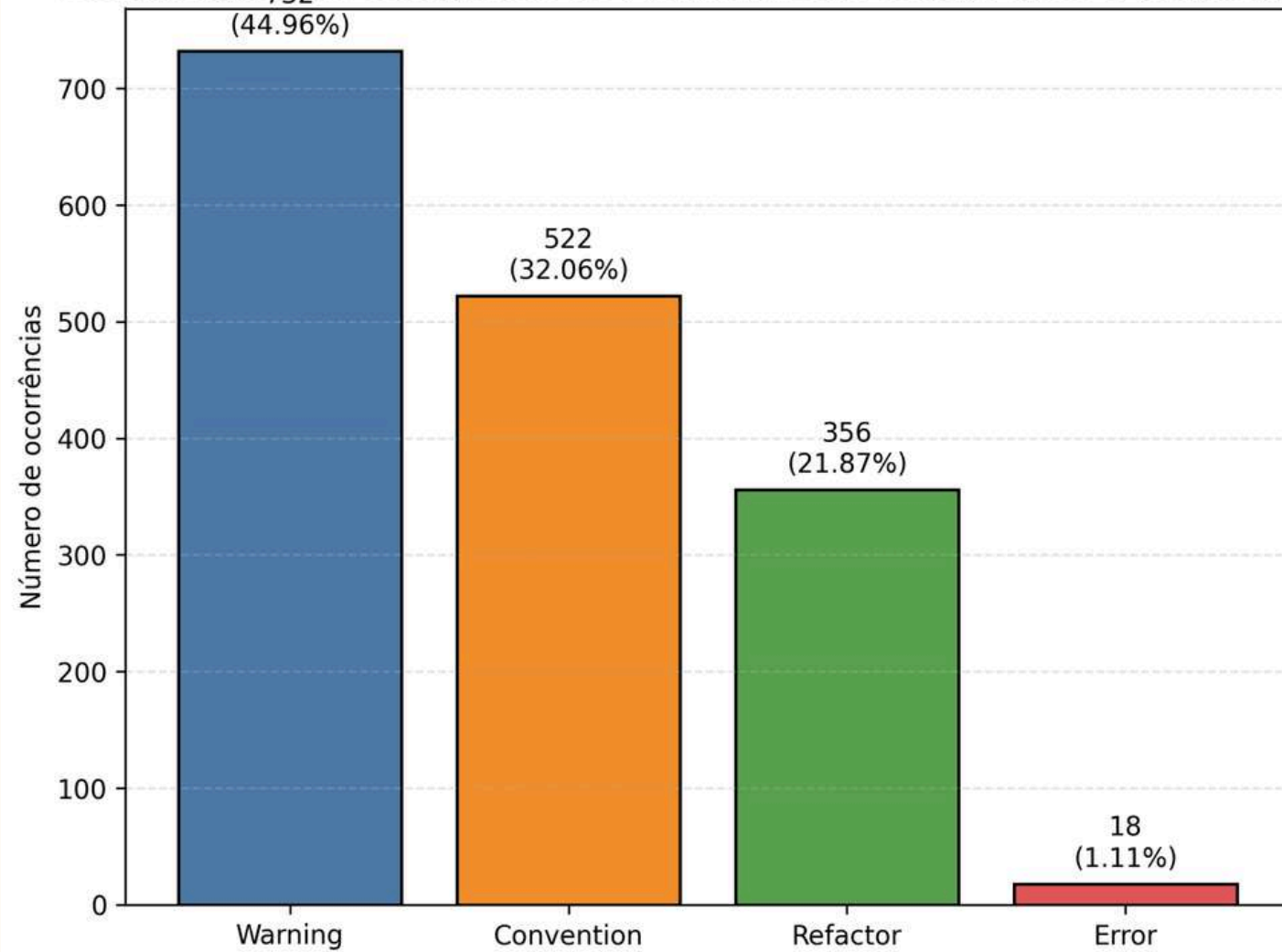
Pós-Refatoração – Sustentabilidade e Testes

- CodeCarbon (Sustentabilidade): O impacto ecológico manteve-se estável. O tempo de execução foi de 1.57s e a energia consumida foi de 6.88E-06 kWh, sem impactos negativos adicionais.
- Pytest (O Grande Desafio):
 - Testes que passaram: 997
 - Testes que falharam: 1.068
- Por que isso aconteceu? A quebra não foi por erros de sintaxe, mas sim pelas profundas alterações estruturais que invalidaram os acoplamentos internos e simulações (mocks) dos testes originais.

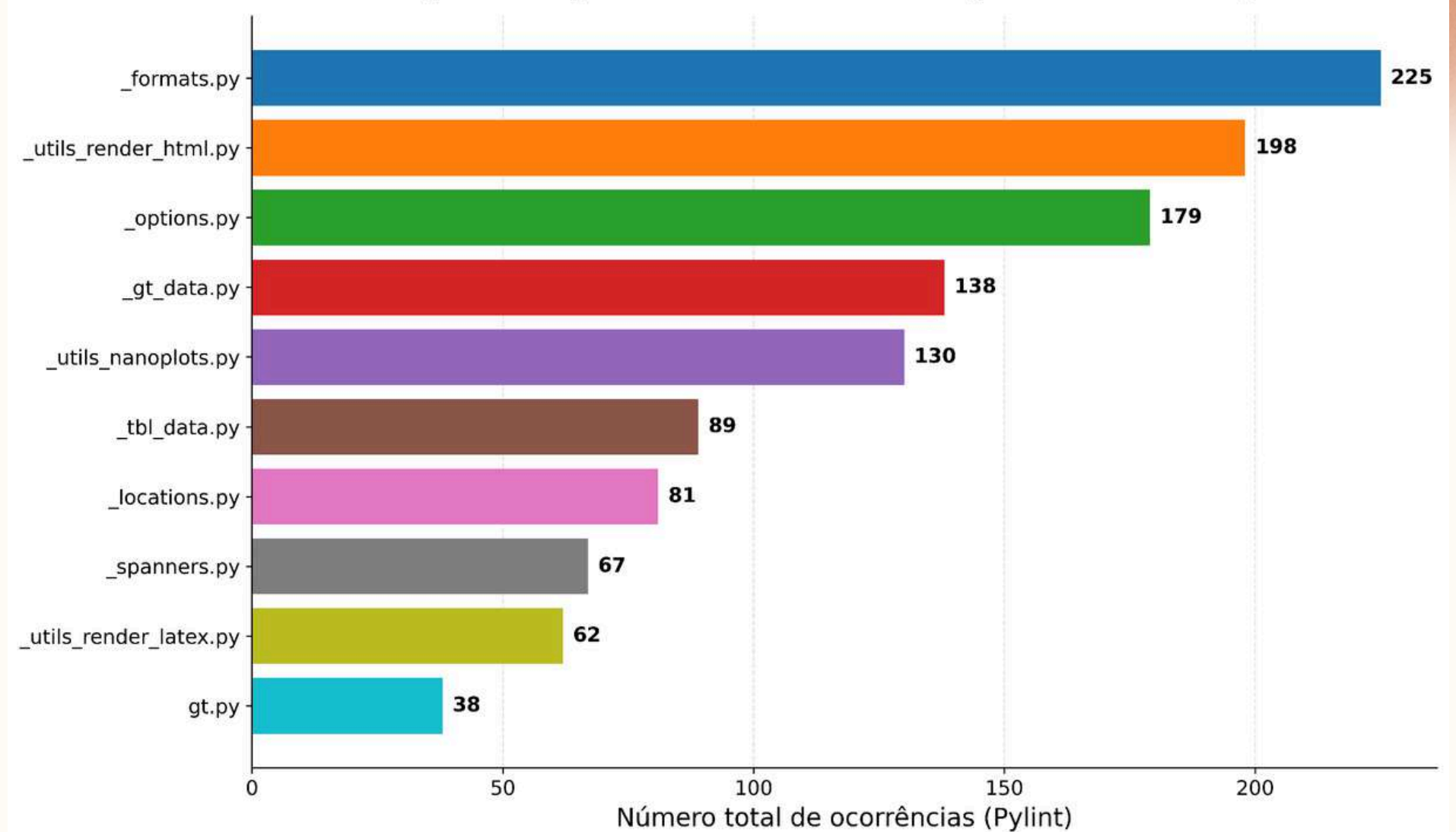


Análise dos Resultados – Pylint (Visão Geral)

Distribuição das Categorias de Problemas de Código após a Refatoração

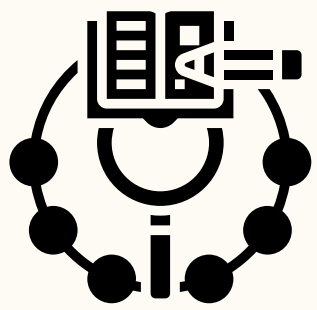


Top 10 arquivos mais críticos após a refatoração

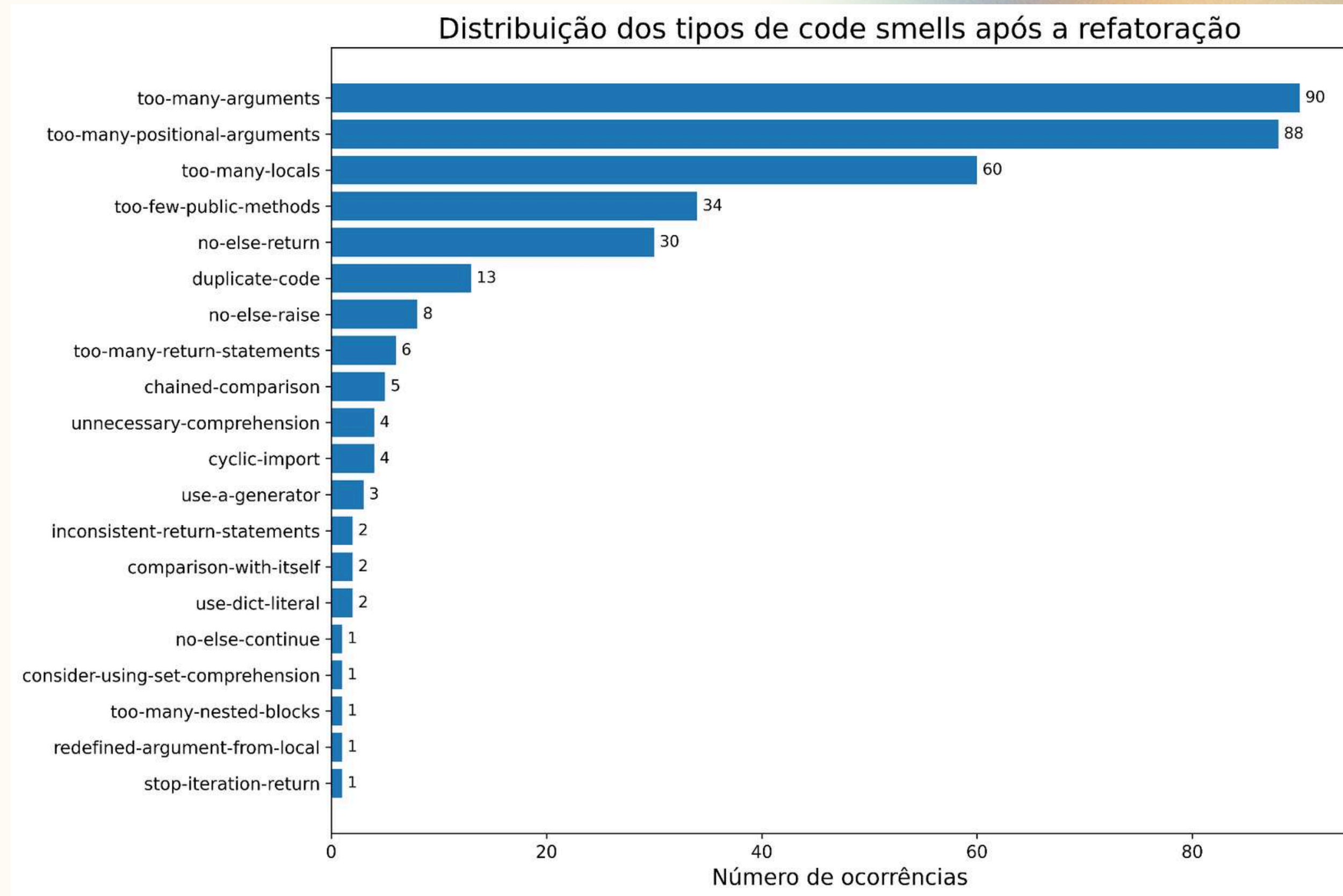


Your code has been rated at

7.80/10



Análise dos Resultados – O Sucesso nos Code Smells





Lições Aprendidas – O Uso da IA na Refatoração

- Acelerador Formidável: O ChatGPT 5.4 Mini foi essencial para modularizar o código, interpretar alertas do Pylint e gerar rapidamente novas funções auxiliares.
- Necessidade de Supervisão: A Inteligência Artificial tenta, por vezes, adotar soluções excessivamente simplistas (como apenas mover blocos de código em vez de repensar a lógica).
- O desenvolvedor humano é crucial para intervir, corrigir a rota e garantir que a refatoração faça sentido arquiteturalmente.



Lições Aprendidas – O Ecossistema Open Source

- Refatorar um projeto real vai muito além de "corrigir sintaxe": exige análise de acoplamento e preocupação constante com a retrocompatibilidade da API.
- A curva de aprendizado para novos contribuidores é acentuada, dificultada pela configuração de ambiente e resolução de dependências de testes locais.

OBRIGADO!

FALE CONOSCO

Wesley: wesleypinheiro@gmail.com

Samuel: fsamuel@alu.ufc.br